

Docker Compose Lab Manual

PostgreSQL Database + .NET Web API with Health Check, depends_on, Network, and Volume

Student Hands-on Lab · From Scratch

Lab Goal

Students will build and run a two-container application using Docker Compose: a PostgreSQL database container and a .NET Web API container. The API waits for the database health check before starting, and both services communicate on a custom Docker network.

What students will learn

Compose file structure, service names, container networking, health checks, depends_on, volumes, environment variables, and useful commands.

Business scenario

A Customer API stores customer records in PostgreSQL. Both services must run together consistently on every developer machine.

1. Prerequisites

- Docker Desktop installed and running
- .NET 8 SDK installed
- Basic terminal knowledge
- A code editor such as Visual Studio 2022 or VS Code

```
docker version
docker compose version
dotnet --version
```

Expected Result

All commands should return version information. If docker compose version fails, update Docker Desktop.

2. Final Architecture

```
Browser / Postman
|
| http://localhost:8080
v
.NET Web API container (service name: api)
|
| Host=postgres;Port=5432
v
PostgreSQL container (service name: postgres)
|
```

v
Docker named volume: postgres-data

Key Concept

Inside Docker Compose, service names become DNS host names. The API connects to the database using Host=postgres, not Host=localhost.

3. Create the Project Folder

```
mkdir compose-postgres-api-lab
cd compose-postgres-api-lab

dotnet new webapi -n CustomerApi --no-openapi
cd CustomerApi
```

4. Add PostgreSQL and EF Core Packages

```
dotnet add package Npgsql.EntityFrameworkCore.PostgreSQL
dotnet add package Microsoft.EntityFrameworkCore.Design
```

5. Add Customer Model

Create a folder named Models and add Customer.cs.

```
mkdir Models

// Models/Customer.cs
namespace CustomerApi.Models;

public class Customer
{
    public int Id { get; set; }
    public string Name { get; set; } = string.Empty;
    public string Email { get; set; } = string.Empty;
}
```

6. Add DbContext

Create a folder named Data and add CustomerDbContext.cs.

```
mkdir Data

// Data/CustomerDbContext.cs
using CustomerApi.Models;
using Microsoft.EntityFrameworkCore;
```

```

namespace CustomerApi.Data;

public class CustomerDbContext : DbContext
{
    public CustomerDbContext(DbContextOptions<CustomerDbContext> options)
        : base(options)
    {
    }

    public DbSet<Customer> Customers => Set<Customer>();
}

```

7. Replace Program.cs

This minimal API includes health endpoints and simple customer insert/read endpoints.

```

using CustomerApi.Data;
using CustomerApi.Models;
using Microsoft.EntityFrameworkCore;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddDbContext<CustomerDbContext>(options =>
    options.UseNpgsql(builder.Configuration.GetConnectionString("DefaultConnection")));

builder.Services.AddHealthChecks();

var app = builder.Build();

app.MapHealthChecks("/health");

app.MapGet("/", () => "Customer API is running");

app.MapGet("/customers", async (CustomerDbContext db) =>
    await db.Customers.ToListAsync());

app.MapPost("/customers", async (Customer customer, CustomerDbContext db) =>
{
    db.Customers.Add(customer);
    await db.SaveChangesAsync();
    return Results.Created($"/customers/{customer.Id}", customer);
});

// For lab simplicity only. In real enterprise applications, use migrations in CI/CD.
using (var scope = app.Services.CreateScope())
{
    var db = scope.ServiceProvider.GetRequiredService<CustomerDbContext>();
    db.Database.Migrate();
}

app.Run();

```

8. Configure appsettings.json

For Docker Compose, the database host is the service name postgres.

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Host=postgres;Port=5432;Database=customerdb;Username=postgres;Password=postgres"
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*"
}
```

9. Create Dockerfile for .NET Web API

Create a file named Dockerfile inside the CustomerApi folder.

```
# Stage 1: Build
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
WORKDIR /src

COPY CustomerApi.csproj .
RUN dotnet restore

COPY . .
RUN dotnet publish -c Release -o /app/publish --no-restore

# Stage 2: Runtime
FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS runtime
WORKDIR /app

COPY --from=build /app/publish .

ENV ASPNETCORE_URLS=http://+:8080
EXPOSE 8080

ENTRYPOINT ["dotnet", "CustomerApi.dll"]
```

10. Create .dockerignore

```
bin/
obj/
.git/
.vs/
.vscode/
*.user
Dockerfile*
docker-compose*.yaml
.dockerignore
```

11. Create EF Core Migration

Run from the CustomerApi folder.

```
dotnet tool install --global dotnet-ef
# If already installed, update it:
# dotnet tool update --global dotnet-ef

dotnet ef migrations add InitialCreate
```

12. Create docker-compose.yml

Go back to the root folder compose-postgres-api-lab and create docker-compose.yml.

```
cd ..
```

```
services:
  postgres:
    image: postgres:16
    container_name: compose-postgres-db
    environment:
      POSTGRES_DB: customerdb
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
    ports:
      - "5432:5432"
    volumes:
      - postgres-data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres -d customerdb"]
      interval: 10s
      timeout: 5s
      retries: 5
      start_period: 10s
    networks:
      - backend-network

  api:
    build:
      context: ./CustomerApi
      dockerfile: Dockerfile
    container_name: compose-customer-api
    environment:
      ASPNETCORE_ENVIRONMENT: Development
      ConnectionStrings__DefaultConnection:
Host=postgres;Port=5432;Database=customerdb;Username=postgres;Password=postgres
    ports:
      - "8080:8080"
    depends_on:
      postgres:
        condition: service_healthy
    networks:
      - backend-network

volumes:
  postgres-data:

networks:
  backend-network:
    driver: bridge
```

Why depends_on with condition?

The API starts only after the postgres service reports healthy. The health check uses pg_isready to verify PostgreSQL is accepting connections.

13. Execute the Lab

```
# Build and start all services
cd compose-postgres-api-lab
docker compose up --build
```

To run in the background:

```
docker compose up --build -d
```

14. Verify Containers

```
docker compose ps
```

Expected: postgres should be healthy and api should be running.

```
docker compose logs postgres
docker compose logs api
```

15. Test the API

```
curl http://localhost:8080/
curl http://localhost:8080/health
```

Create a customer:

```
curl -X POST http://localhost:8080/customers -H "Content-Type: application/json" -d
{"name": "Gayathri", "email": "gayathri@example.com"}
```

Read customers:

```
curl http://localhost:8080/customers
```

16. Optional: Check PostgreSQL Directly

```
docker exec -it compose-postgres-db psql -U postgres -d customerdb
```

```
\dt
SELECT * FROM "Customers";
\q
```

17. Compose File Explanation

Compose Section	Meaning
services	Defines the containers to run.
postgres	The PostgreSQL database service.
api	The .NET Web API service.
environment	Passes configuration values into containers.
ports	Maps host machine ports to container ports.
volumes	Persists PostgreSQL data outside the container.
healthcheck	Checks whether PostgreSQL is ready.
depends_on	Starts the API only after PostgreSQL is healthy.
networks	Places both services on the same Docker network.

18. Troubleshooting Guide

API cannot connect to database

Check the connection string. Inside Compose use Host=postgres, not Host=localhost.

postgres is unhealthy

Run docker compose logs postgres and verify username, password, and database name.

Port already in use

Change the left side of the port mapping, for example 5433:5432 or 8081:8080.

Migration fails

Check that the EF migration folder exists and the API has Npgsql package installed.

Data not deleted after compose down

Named volumes persist by default. Use docker compose down -v only when you want to delete data.

19. Useful Docker Compose Commands

Command	Purpose
docker compose up --build	Build and start services in foreground.
docker compose up --build -d	Build and start services in background.
docker compose ps	Show service status.
docker compose logs api	Show API logs.
docker compose logs postgres	Show database logs.
docker compose restart api	Restart only API service.
docker compose down	Stop and remove containers and network. Keeps volumes.
docker compose down -v	Stop and remove containers, network, and volumes. Deletes database data.

20. Student Assignment

1. Change the API external port from 8080 to 8081 and test it.
2. Add another customer using curl or Postman.
3. Run docker compose down and then docker compose up -d. Verify data still exists.
4. Run docker compose down -v and explain why data is gone.

5. Change the database password and update the API connection string accordingly.

21. Knowledge Check

Q: Why does the API use Host=postgres?

A: Because postgres is the Docker Compose service name and Compose DNS resolves it.

Q: What does the PostgreSQL health check do?

A: It verifies that PostgreSQL is ready to accept connections.

Q: What does depends_on condition: service_healthy mean?

A: The API service waits until the database health check passes before starting.

Q: What happens to data when docker compose down is used?

A: Containers and network are removed, but named volumes remain.

Q: What command deletes database data?

A: docker compose down -v